CSS layout

CSS layout

- The 3 core concepts to understand about CSS layout are:
    1. The CSS box model
    2. Floating
    3. Positioning
- These 3 concepts control the way elements are arranged and displayed on a page.

CSS Box Model

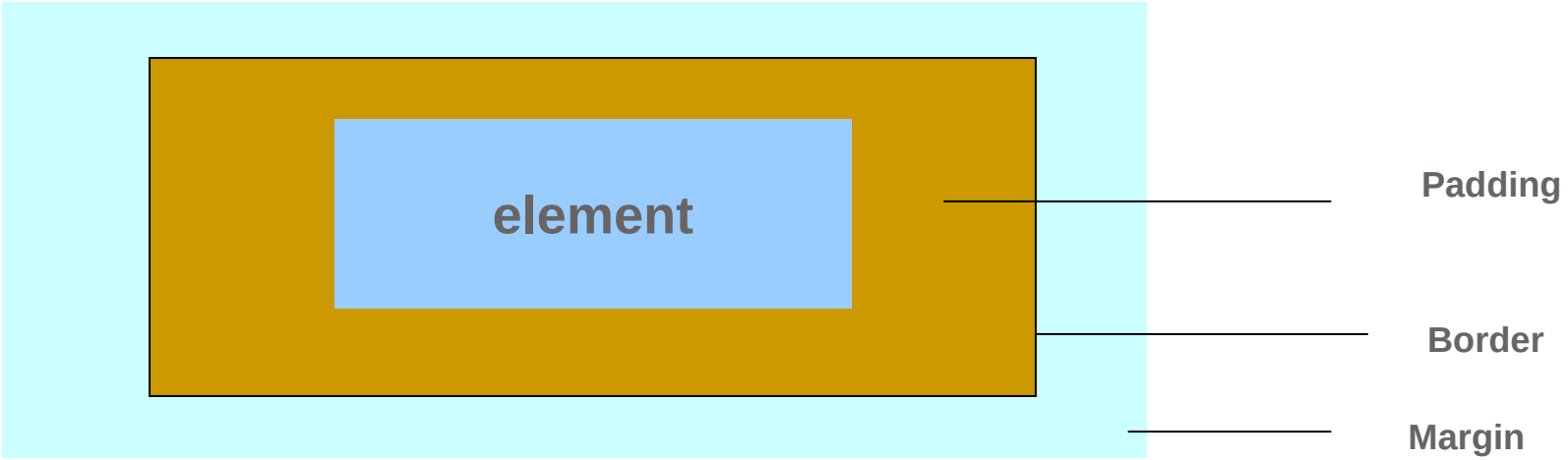


CSS Box Model

- When a browser draws an object on a page, it places it into an invisible rectangular space called a "bounding box."
- You can specify the size, look, and feel of the **margins**, the **padding**, the **border**, and the contents of that bounding box.
- *Internet Explorer* interprets CSS box styles differently than most other web browsers.

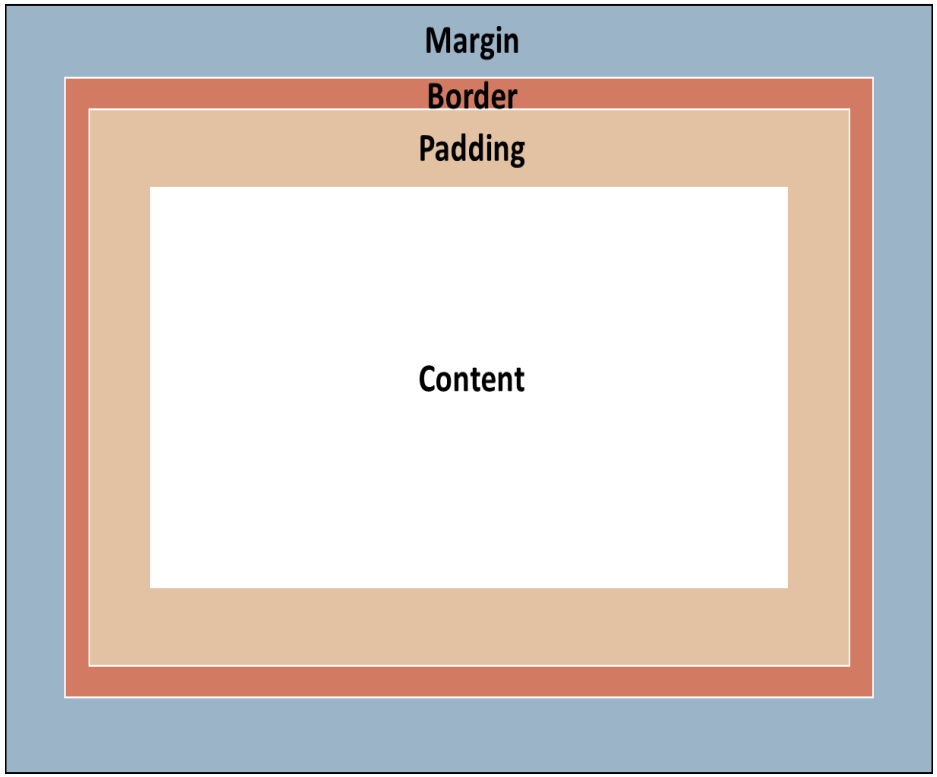
CSS: Box Model

The margin is the area outside of the element. It never has color; it is always transparent.
The border extends around the element, on the outer edge of any padding. The border can be of several types, widths, and colors.
The padding exists around the content and inherits the background color of the content area.
The content is surrounded by padding.



The CSS Box Model

- The Box Model control the way elements are arranged and displayed on a page.
- Every block element in CSS is divided into regions consisting:
    1. Content
    2. Padding
    3. Border
    4. Margins



Understanding the Box Model

- The CSS box model describes the boxes that are formed around elements in a Web page
  - The content area is the area that contains the box content
  - The box properties are used to add white space and a border around the content
  - Padding is the white space that surrounds the box content
  - A border can be placed around the padding
  - The margin is white space outside the border

The CSS Box Model

- Every element in HTML is considered a "box," whether it is a paragraph, a , an image, or so on.
- Boxes have consistent properties.
- The box model describes the way in which every HTML **block-level element** has the potential for a border, padding, and margin and how the border, padding, and margin are applied.

 gkkkg

 gkkkg

The box model describes the way in which every HTML **block-level element** has the potential for a border, padding, and margin and how the border, padding, and margin are applied.

Understanding the Box Model

- The CSS box model describes the boxes that are formed around elements in a Web page
  - The content area is the area that contains the box content
  - The box properties are used to add white space and a border around the content
  - Padding is the white space that surrounds the box content
  - A border can be placed around the padding
  - The margin is white space outside the border

Settings

- The core border properties are:
  - **Width:** absolute (px, in, cm, or 'thin', 'medium', 'thick'), or relative (ems)
  - **Style:** dotted, dashed, solid, double, groove, ridge, inset, outset, hidden, etc
  - **Color:** '**blue**', '**red**', **#FF9900**, etc
- You can also create the effect of a border by using a graphic image in a CSS background property, instead of the border property

Margins & Padding

- **Margins**
- Margins define the space around elements outside the border
- Margin properties can have negative values in order to deliberately overlap content
- Margin properties will affect the position of background elements (graphics and/or colours) in relation to the edges of the containing block element
- Margin properties can be defined independently on top, right, bottom and left, or all-at-once using CSS shorthand

Margins & Padding

- **Padding**
- Padding defines the space around elements inside the border; i.e between the border and the content itself
- Padding properties cannot have negative values
- Padding properties do not affect the position of background elements (graphics and/or colors) in the containing block element; only the position of content.
- Padding properties can be defined independently on top, right, bottom and left, or all-at-once using CSS shorthand

Margin

- the margin property works with space outside of the element.
- Following are the style properties for setting margins:
  - **margin-top**—Sets the top margin
  - **margin-right**—Sets the right margin
  - **margin-bottom**—Sets the bottom margin
  - **margin-left**—Sets the left margin
  - **margin**—Sets the top, right, bottom, and left margins as a single property

How to set Margin

- To set a margin of 15 pixels on all sides.
- **First method:**

```
margin-top:15px;
margin-right:15px;
margin-bottom:15px;
margin-left:15px;
```

- **Second method**: short hand form:

```
margin:15px 15px 15px 15px;
```

Three Short hands for Setting Margins, Padding or Borders

1. **One value**—The size of all the margins
2. **Two values**—The size of the top/bottom margins and the left/right margins (in that order)
3. **Four values**—The size of the top, right, bottom, and left margins (in that order)

- To set a margin of 15 pixels on all sides.
- margin:15px;
- margin:15px 15px;
- margin:15px 15px 15px 15px;

Example1: Simple Code to Produce Four Color s with Borders and Margins

```
<html>
<head>
<title>Color Blocks</title>
<style type="text/css">
body {
margin:0px;
}
div {
width:250px;
height:100px;
border:5px solid #000000;
color:black;
font-weight:bold;
text-align:center;
}
```

Example1: Simple Code to Produce Four Color s with Borders and Margins (Cont.)

```
Div#d1 {
background-color:red;
margin:15px;
}
div#d2 {
background-color:green;
margin:15px;
}
div#d3 {
background-color:blue;
}
div#d4 {
background-color:yellow;
margin:15px;
}
</style>
</head>
```

Example1: Simple Code to Produce Four Color s with Borders and Margins (Cont.)

```
<body>
<div id="d1">DIV #1</div>
<div id="d2">DIV #2</div>
<div id="d3">DIV #3</div>
<div id="d4">DIV #4</div>
</body>
</html>
```

Modify the above **CSS** code to produce the following:

1. No margin around the first color block.
2. A left-side margin of 15 pixels, a top margin of 5 pixels, and no bottom margin around the second color block.
3. A left-side margin of 75 pixels and no top margin or bottom margins around the third color block.
4. A left-side margin of 250 pixels and a top margin of 25 pixels around the fourth color block.

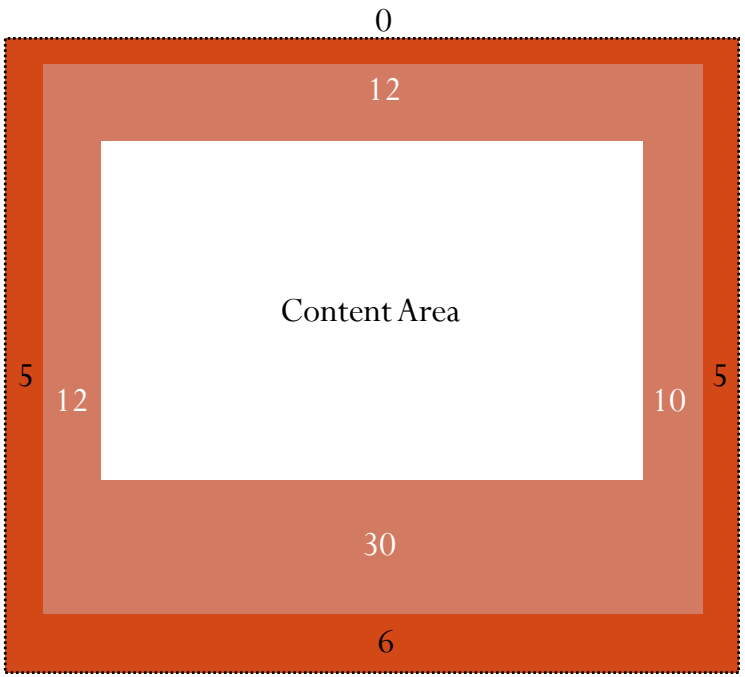
Example 2: changing the style of example 1

```
<html>
<head>
<title>Color Blocks</title>
<style type="text/css">
body {
margin:0px;
}
div {
width:250px;
height:100px;
color:black;
font-weight:bold;
text-align:center;
}
div#d1 {
border:5px solid #000000;
background-color:red;
margin:0px;
}
div#d2 {
border-width:6px 6px 3px 6px;
border-style:solid;
border-color:#000000;
background-color:green;
margin:5px 0px 0px 15px;
}
```

```
div#d3 {
border-width:3px 6px 6px 6px;
border-style:solid;
border-color:#000000;
background-color:blue;
margin:0px 0px 0px 75px;
}
div#d4 {
border:5px solid #000000;
background-color:yellow;
margin:25px 0px 0px 250px;
}
</style>
</head>
<body>
<div id="d1">DIV #1</div>
<div id="d2">DIV #2</div>
<div id="d3">DIV #3</div>
<div id="d4">DIV #4</div>
</body>
</html>
```

Exercise: CSS Margin & Padding

```
#container {
        margin-top: 0;
        margin-right: 5px;
        margin-bottom: 6px;
        margin-left: 5px;
        padding-top: 20px;
        padding-right: 10px;
        padding-bottom: 30px;
        padding-left: 12px;
}
```



CSS Shorthand: Margin & Padding

```
#container {
        padding: 20px 10px 30px 12px;
        margin: 0px 5px 6px 5px;
}
```

The Padding Element

- **Padding** is similar to **margins** in that it adds extra space to elements, but the big difference is where that space is located.
  - Margins are added to the **outside** of elements.
  - Padding adds space ***inside*** the rectangular area of an element.

  - Example, if you create a style rule for an element that establishes a width of 50 pixels and a height of 30 pixels, and then sets the padding of the rule to 5 pixels, the remaining content area will be 40 pixels by 20 pixels.
  - Also, because the padding of an element appears within the element's content area, it will assume the same style as the content of the element, including the background color.

How to set Padding

- **padding-top**—Sets the top padding
- **padding-right**—Sets the right padding
- **padding-bottom**—Sets the bottom padding
- **padding-left**—Sets the left padding
- **padding**—Sets the top, right, bottom, and left padding as a single property

Three Short hands for Setting Margins, Padding or Borders

1. **One value**—The size of all the margins
2. **Two values**—The size of the top/bottom margins and the left/right margins (in that order)
3. **Four values**—The size of the top, right, bottom, and left margins (in that order)

- To set a padding of 20 pixels on all sides.
- padding:20px;
- padding:20px 20px;
- padding:20px 20px 20px 20px;

Simple Code to Produce Four Colored s with Borders, Margins, and Padding

```
<html>
<head>
<title>Color Blocks</title>
<style type="text/css">
body {
margin:0px;
}
div {
width:250px;
height:100px;
border:5px solid #000000;
color:black;
font-weight:bold;
margin:25px;
}
```

```
div#d1 {
background-color:red;
text-align:center;
padding:15px;
}
div#d2 {
background-color:green;
text-align:right;
padding:25px 50px 6px 6px;
}
div#d3 {
background-color:blue;
text-align:left;
padding:6px 6px 6px 50px;
}
```

Simple Code to Produce Four Color s with Borders, Margins, and Padding (Continued)

```
div#d4 {
background-color:yellow;
text-align:center;
padding:50px;
}
</style>
</head>
<body>
<div id=”d1”>DIV #1</div>
<div id=”d2”>DIV #2</div>
<div id=”d3”>DIV #3</div>
<div id=”d4”>DIV #4</div>
</body>
</html>
```

CSS Float



Understanding the float property

- The float property allows elements to be moved around in the design such that other elements can wrap around them.
- used in conjunction with images, etc.
- Elements float horizontally, not vertically, thus, two possible values: right and left.
- example,
  - if you have three s float values of left, they will all line up to the left of the containing body element.
  - If you have your s within another , they will line up to the left of that element, even if that element itself is floated to the right.

Floats

- Syntax:

```
float: position;
```

- where *position* is left, right, or none.

- Setting the float property to left or right causes a box to be ***taken out of its position in the normal flow*** and moved as far left or right as possible.



Float Values

- The Float property has three value options:
- **float: left;**
- **float: right;**
- **float: none;**



Restoring the Normal Flow: “Clear”

- To restore the “normal flow”, we can use the clear property.
- The clear property has three value options:
- clear: left;
- clear: right;
- clear: both;
- These specify which side of the element’s box may ***not*** have a float next to it.



Example1: Using float to Place s

```
<html>
<head>
<title>Color Blocks</title>
<style type=”text/css”>
body {
margin:0px;
}
div {
width:250px;
height:100px;
border:5px solid #000000;
color:black;
font-weight:bold;
margin:25px;
}
div#d1 {
background-color:red;
float:left;
}
```

```
div#d2 {
background-color:green;
float:left;
}
div#d3 {
background-color:blue;
float:left;
}
div#d4 {
background-color:yellow;
float:right;
}
</style>
</head>
<body>
<div id=”d1”>DIV #1</div>
<div id=”d2”>DIV #2</div>
<div id=”d3”>DIV #3</div>
<div id=”d4”>DIV #4</div>
</body> </html>
```

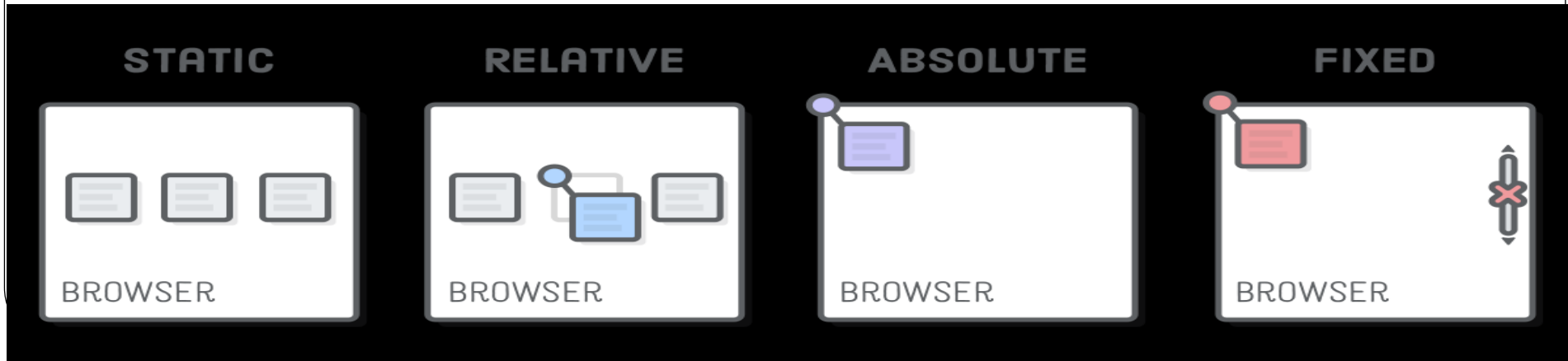
CSS Positioning (CSS-P)

- **CSS-P was released by W3C in 1997**
- **Completely supported by IE9, FX3, and Chrome**



CSS Positioning

- The third core concept to understand in CSS layout (after the 'box model' and 'floats'), is positioning.
- Element Positioning:
  - let you manually position elements using specific Coordinates.
  - advanced positioning lets you place element where ever on the page.
  - E,g, "Put that box 20 pixels above and 50 pixels to the right of its parent's origin."
- The possible values of position are:



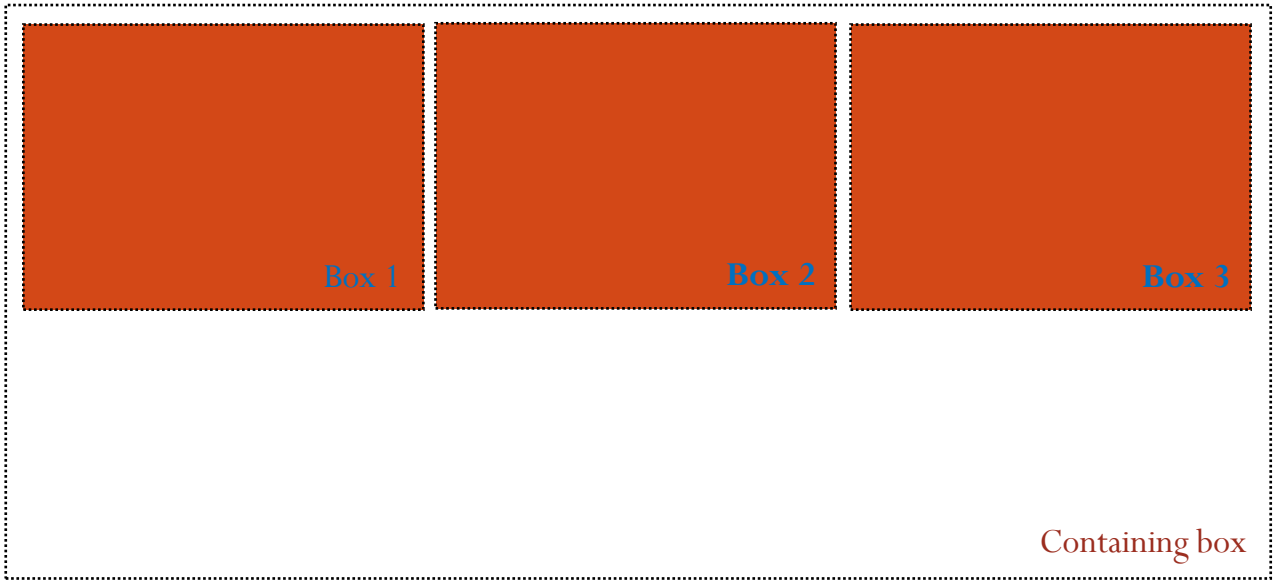
CSS Positioning

- There are different types of positioning that can be applied to CSS boxes:
  - **Relative Positioning**
  - **Absolute Positioning**
  - **Static Positioning**
  - **Fixed Positioning**
- Understanding the differences between the two is difficult at first, but important!

property	value	description
position	static	default position
	relative	offset from its normal static position
	absolute	a fixed position within its containing element
	fixed	a fixed position within the browser window
top, bottom, left, right	positions of box's corners	

CSS Positioning: Relative Positioning

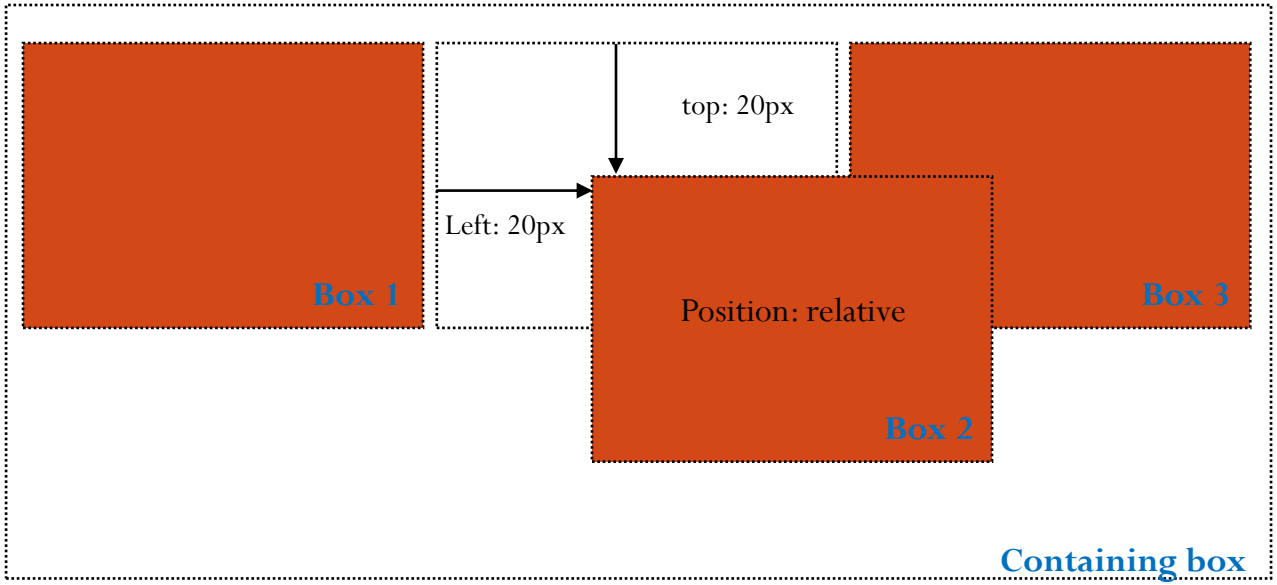
- A relatively positioned element will stay exactly where it is, in relation to the normal flow.
- You can then offset its position "relative" to its starting point in the normal flow:



CSS Positioning: Relative Positioning

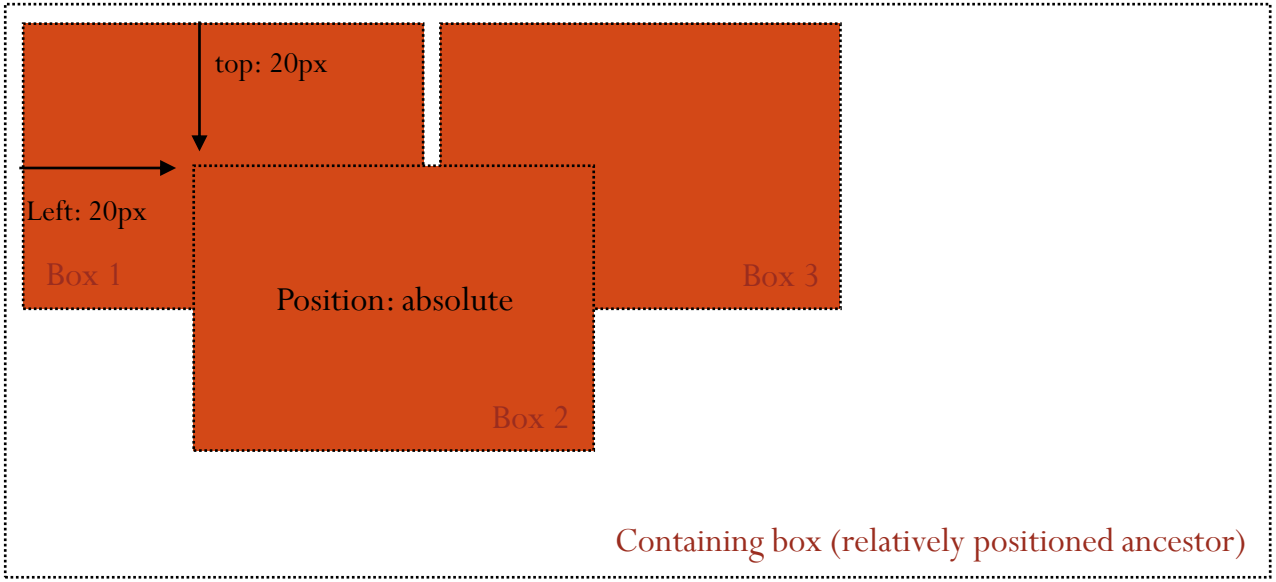
- In this example, **box 2** is offset 20px, top and left. The result is the box is offset 20px from its original position in the normal flow.
- Box 2 may overlap other boxes in the flow, but other boxes still recognise its original position in the flow.

```
#myBox {
    position: relative;
    left: 20px;
    top: 20px;
}
```



CSS Positioning: Absolute Positioning

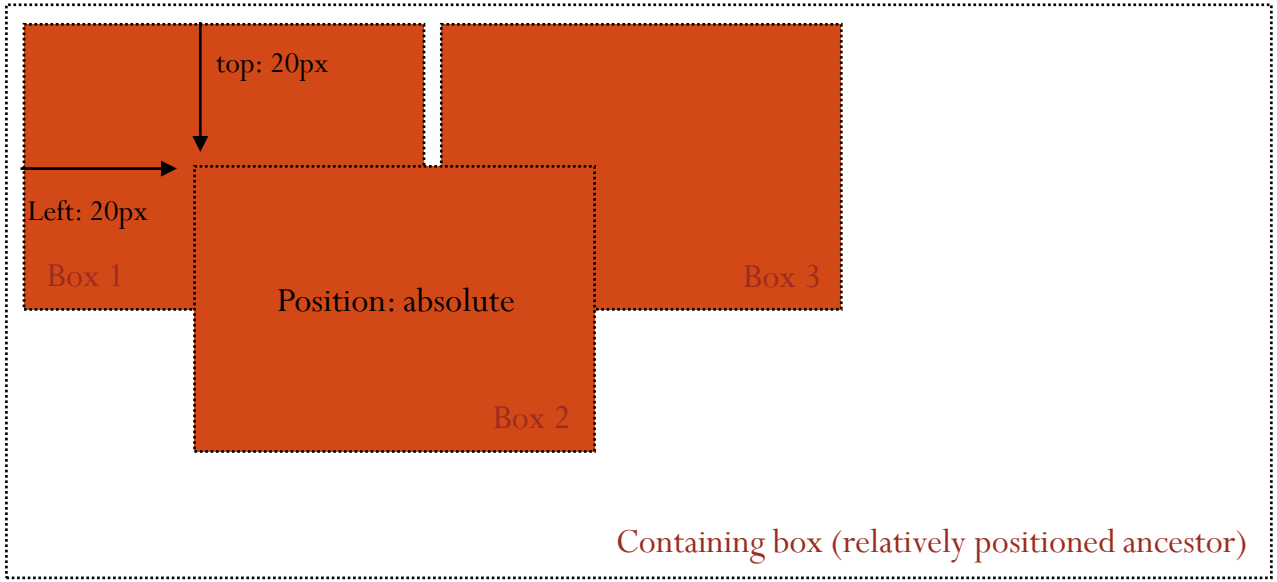
- An absolutely positioned box is taken out of the normal flow, and positioned in relation to its nearest positioned ancestor (i.e. its containing box).
- If there is no ancestor box, it will be positioned in relation to the initial containing block, usually the browser window.



CSS Positioning: Absolute Positioning

- An absolutely positioned box can be offset from its initial position inside the containing block, but other boxes within the block (and still within the normal flow) act as if the box wasn't there.

```
#myBox {
    position: absolute;
    left: 20px;
    top: 20px;
}
```



CSS Positioning: Fixed Positioning

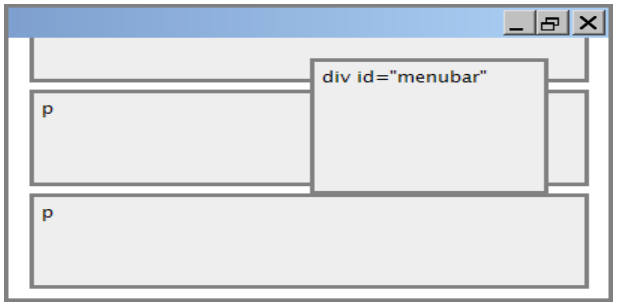
- Fixed Positioning is a sub-category of Absolute Positioning
- Allows the creation of floating elements that are always fixed in the same position in the browser window, while the rest of the content scrolls as normal
- (rather like the effect of fixed background attachments)
- PROBLEM: fixed positioning is not supported in IE5 and IE6(!), but can be made to work with JavaScript for those browsers

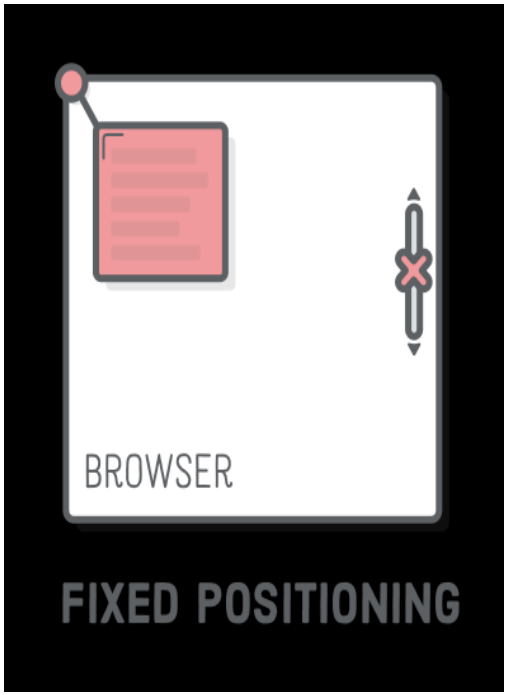
```
Div #ad {
  position: fixed;
  right: 10%;
  top: 45%;
}

CSS
```

Fixed positioning

- "Fixed positioning" has a lot in common with absolute positioning:
- the element is removed from the normal flow of the page, and the coordinate system is relative to the entire browser window. The key difference is that fixed elements don't scroll with the rest of the page.
- removed from normal flow (like floating ones)
- positioned relative to the browser window
  - even when the user scrolls the window, element will remain in the same place

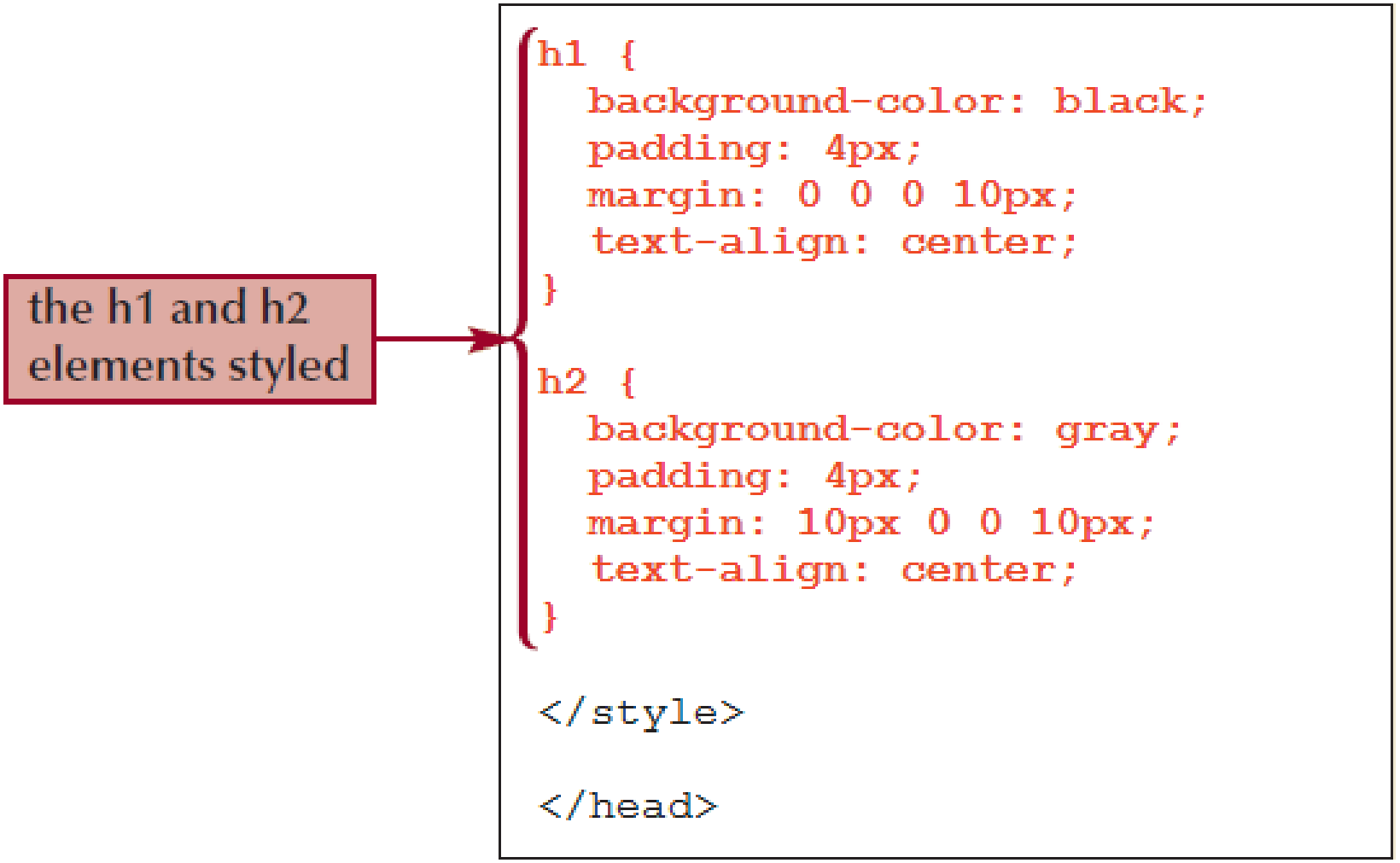




Floats & Positioning

- **Summary:**
- Floats (also a form of positioning) takes boxes out of the normal flow and "floats" them left or right edges of the containing block
- Relative Positioning is "relative" to the element's initial position in the normal flow.
- Absolute Positioning is "relative" to the nearest positioned ancestor, or (if one doesn't exist) the initial container block (usually the browser window)
- Fixed Positioning is fixed in one position "relative" to the browser window only — does not scroll with content (Not supported in IE5, IE6)

Internal Style sheet



External Style Sheets

- Save the CSS file with a **.css** filename extension.
- In the <head> section of the XML file, enter the following code to link to the external CSS file:

```
<link rel="stylesheet" href="default.css"
type="text/css">
```

where *filename.css* is the name of the external style sheet file.

End

The CSS Box Model

```
div {
width: 250px;
height: 100px;
background-color: #ff0000;
border: 1px solid #000000;
}
div#d2 {
width: 250px;
height: 100px;
background-color: #ff0000;
border: 5px solid #000000;
margin: 10px;
padding: 10px;
}
```

How to justify the content with CSS

The HTML attribute is used to justify-content is align attribute. The align attribute of <p> tag is used to justify the text on a web page. This can be done by assigning the value to the aligned attribute as justified.

Syntax

text-align: start;
text-align: end;
text-align: left;
text-align: right;
text-align: center;
text-align: justify;

Example

```
<!DOCTYPE html>
<html>

<head>
        <title>Justified content</title>
</head>

<body>
<h3>The content is justified</h3>

<p align="justify">
Welcome to Geeks for Geeks. It is a
computer science portal for geeks.
It contains well written, well thought articles.
We are learning how to justify content on a
web page.
</p>
</body>

</html>
```

3.11 Background Images

- The `background-image` property

→ SHOW `back_image.html` and display

- Repetition can be controlled

  - `background-repeat` property

    - Possible values: `repeat` (default), `no-repeat`, `repeat-x`, or `repeat-y`

  - `background-position` property

    - Possible values: `top`, `center`, `bottom`, `left`, or `right`

```
<style>
body {
  background-image: url('img_girl.jpg');
  background-repeat: no-repeat;
  background-attachment: fixed;
  background-size: cover;
}
</style>
```

```
Background Stretch
Example
<style>
body {
  background-image: url('img_girl.jpg');
  background-repeat: no-repeat;
  background-attachment: fixed;
  background-size: 100% 100%;
}
</style>
```

```
<p align="justify">
```

Span and Div

- There are two tags that are particularly useful when using CSS: `<span>` and `<div>`. They are both container tags that have minimal formatting associated with them.
- The `<span>` tag is an **inline element** that simply holds text without doing anything special to it.
- The `<div>` tag is a **block element** and causes the text it encloses to start on a new line.
- Using `<span>` and `<div>` tags in conjunction with classes and IDs allows for great flexibility in creating pages.

Example: SPAN, DIV, Class, and ID

- Here's an example of a web page using a class, an id, and the span and div tags:

```
<head>
<style type="text/css">
<!--
#BrightText {color: #00ff00; font-weight: bolder;}
.PeriwinkleBlueText {color: #6699ff;}
-->
</style>
</head>

<html>
<body>
Baa, baa black sheep <br />
Have you any wool?<br />

<div id="BrightText">
Yes sir!  Yes sir!
</div>

Three bags full.  <br />
<span class="PeriwinkleBlueText">One</span> for my master.  <br />
<span class="PeriwinkleBlueText">One</span> for my dame.  <br />
<span class="PeriwinkleBlueText">One</span> for the little boy,  <br />
Who lives down the lane.
</body>
</html>
```

Baa Baa Black Sheep
Have you any wool?
Yes sir! Yes sir!
Three bags full.
One for the master
One for the dame
One for the little boy
Who lives down the lane.

3.12 The and tags

- One problem with the font properties is that they apply to whole elements, which are often too large

  - Solution: a new tag to define an element in the content of a larger element -

  - The default meaning of is to leave the content as it is

```
<p>
Now is the <span> best time </span> ever!
</p>
```

  - Use to apply a document style sheet to its content

```
<style type = "text/css">
  .bigred {font-size: 24pt;
           font-family: Ariel; color: red}
</style>

    ...

<p>
   Now is the
       <span class = "bigred">
   best time </span> ever!
</p>
```

3.12 The `<span>` and `<div>` tags (continued)



- The `<span>` tag is similar to other HTML tags, they can be nested and they have `id` and `class` attributes

- Another tag that is useful for style specifications:
`<div>`

- Used to create document sections (or divisions) for which style can be specified

- e.g., A section of five paragraphs for which you want some particular style

CSS Float

- If you want to wrap content around other content (such as text around a picture), you can use the `float` property.

- The `float` property determines on which side of the bounding box the element aligns so that the other content wraps around it.

3.10 CSS Float

- Margin – the space between the border of an element and its neighbor element

- The margins around an element can be set with `margin-left`, etc. - just assign them a length value

```
<img src = "c210.jpg " style = "float: right;
   margin-left: 0.35in;
   margin-bottom: 0.35in" />
```



This is a picture of a Cessna 210. The 210 is the flagship single-engine Cessna aircraft. Although the 210 began as a four-place aircraft, it soon acquired a third row of seats, stretching it to a six-place plane. The 210 is classified as a high performance airplane, which means its landing gear is retractable and its engine has more than 200 horsepower. In its first model year, which was 1960, the 210 was powered by a 260 horsepower fuel-injected six-cylinder engine that displaced 471 cubic inches. The 210 is the fastest single-engine airplane ever built by Cessna.

Positioning

- Using CSS, you can place elements exactly on a page using a technique called "positioning." Positioning is determined by an X axis and Y axis. To specify a point on the screen, you can use the X and Y coordinates for that point.
- There are several ways to specify position in CSS: *absolute*, *relative*, *fixed*, *inherit*, and *static*.
- The three most often used are *absolute*, *relative*, and *fixed*.
- *Internet Explorer 6* only recognizes *absolute* and *relative* positioning.
- *Internet Explorer 7* recognizes *fixed* positioning in standards-compliance (strict HTML/XHTML) mode.

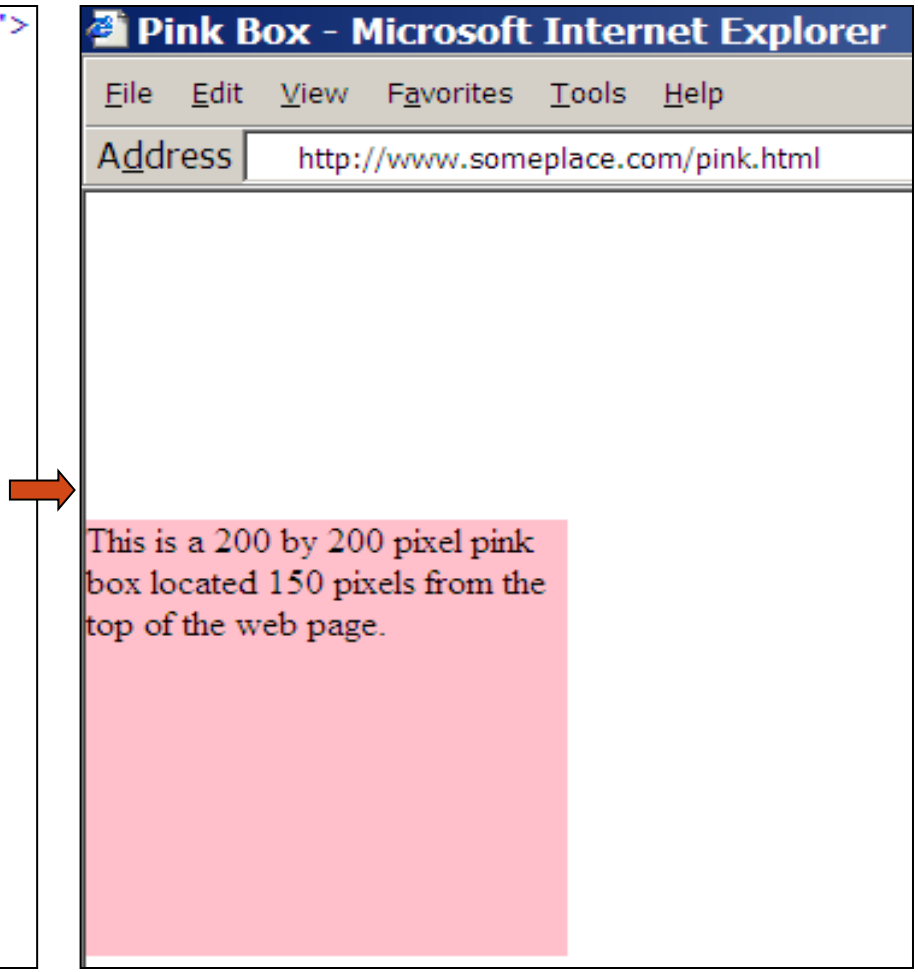
Absolute, Relative, Fixed, Inherit, and Static Positioning

- *Absolute positioning* defines the position of a given bounding box from the top and left side margins of the web page. This not only allows objects to be placed in an exact location, it also allows objects to be placed one on top of another.
- *Relative positioning* defines the positioning in such a way that elements are offset from the previous element in the HTML code. This allows objects to be placed in relation to one another.
- *Fixed positioning* defines the position of a given box relative to the window and remains in its specified location even as the content scrolls underneath it. This value does not work in Internet Explorer 6 or earlier. In IE 7, the browser must be in "standards-compliance mode".
- *Inherit positioning* explicitly sets the value to that of the parent (if the parent is *position:absolute*, the child will be *position:absolute*; if the parent is *position:fixed*, the child will be *position:fixed*).
- *Static positioning* is the default. It defines the position of a given box essentially as an unpositioned element – it flows in the normal rendering sequence of the web page.

Example: position:absolute

```
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<title>Pink Box</title>
<style type="text/css">
<!--
#pinkbox {
    position: absolute;
    width: 200px;
    background-color: pink;
    left: 0px;
    top: 150px;
    height: 200px;
    }
-->
</style>
</head>

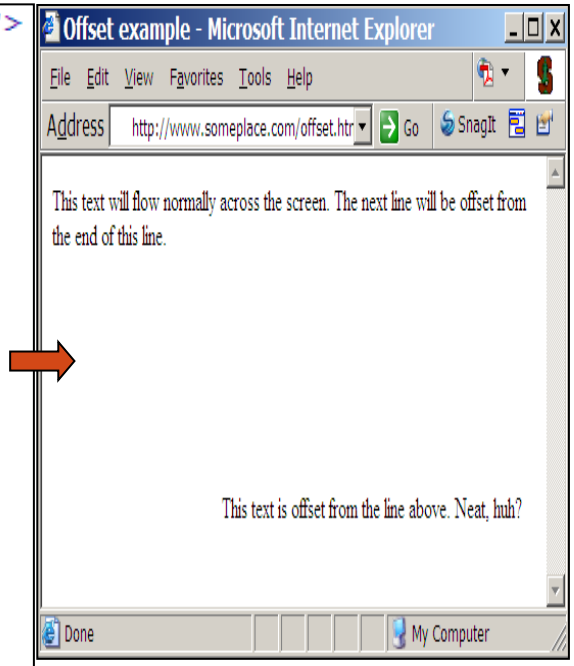
<body>
<div id="pinkbox">This is a 200
by 200 pixel pink box located
150 pixels from the top of the
web page.
</div>
</body>
</html>
```



Example: position:relative

```
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<title>Offset example</title>
<style type="text/css">
<!--
.offset {
    position: relative;
    top: 150px;
    left: 50px;
    }
-->
</style>
</head>

<body>
<p>This text will flow normally across
the screen.  The next line will be offset
from the end of this line.
<span class="offset">This text is offset
from the line above.  Neat, huh?</span>
</p>
</body>
</html>
```



Example: position:fixed (Code View)

```
<!DOCTYPE html>
<html>
<head>
<title>Untitled Document</title>
<style type="text/css">
<!--
#links {
   position:fixed;
   border:dotted;
   border-color:#000000;
   width:20%;
   height:100%;
   z-index:1;
   left: 0px;
   top: 0px;
   background-color: #FFFFCC;
}
#main {
   position:absolute;
   left:25%;
   top:0px;
   width:70%;
   }
-->
</style>
</head>
```

```
<div id="main">
  <p>Lorem ipsum dolor sit amet, consectetuer adipiscing
    elit. Quisque ultrices,  nibh ac rhoncus fermentum,
    orci sem dapibus nisi, sed tincidunt lectus lectus at
    augue. In consectetuer vehicula enim. In hac habitasse
    platea dictumst. Donec a  nisl vitae tortor tristique
    viverra. Sed at lorem a ante lobortis molestie.  Nulla
    ullamcorper urna accumsan diam. Aliquam non eros.
    Pellentesque egestas  ultricies enim. Aenean lobortis.
    Nulla interdum commodo turpis. Sed ut mi id  elit
    vehicula sollicitudin. Sed lobortis, ligula sit amet
    euismod egestas, mi  ante iaculis nunc, ut rhoncus
    magna lectus ac arcu. In hac habitasse platea
    dictumst. Proin quis ligula vitae quam pharetra
    adipiscing. Pellentesque  tincidunt suscipit nibh. Ut
    fermentum suscipit justo. </p>

  <p>Fusce purus lectus, ultricies nec, aliquam at,
    facilisis id, arcu. Vestibulum  quis mi vel massa
    porta hendrerit. Nulla ullamcorper ligula nec lectus.
    Quisque  tempor, augue in molestie gravida, eros arcu
    luctus tortor, eu dignissim diam  urna sed urna. Ut
    dictum ultrices lacus. In hac habitasse platea
    dictumst.  Suspendisse sed purus blandit metus
    ultricies suscipit. Proin diam justo,  consequat id,
    rhoncus eget, facilisis ut, lacus. Vivamus dignissim
    dui in justo.  Suspendisse elit. Nam nulla tortor,
    fringilla sed, faucibus quis, ullamcorper a,  leo.
    Fusce blandit condimentum turpis. Pellentesque vel odio
    et odio suscipit  egestas. Nullam ullamcorper sagittis
    ipsum. Maecenas fringilla malesuada pede.  Duis ut
    quam. </p>

</div>

<div id="links">

  <p>This area is fixed and will never move. It's good for
    things like navigation bars.</p>

  <ul>

    <li><a href="page1.html">Page 1</a></li>

    <li><a href="page2.html">Page 2</a></li>

    <li><a href="page3.html">Page 3</a></li>

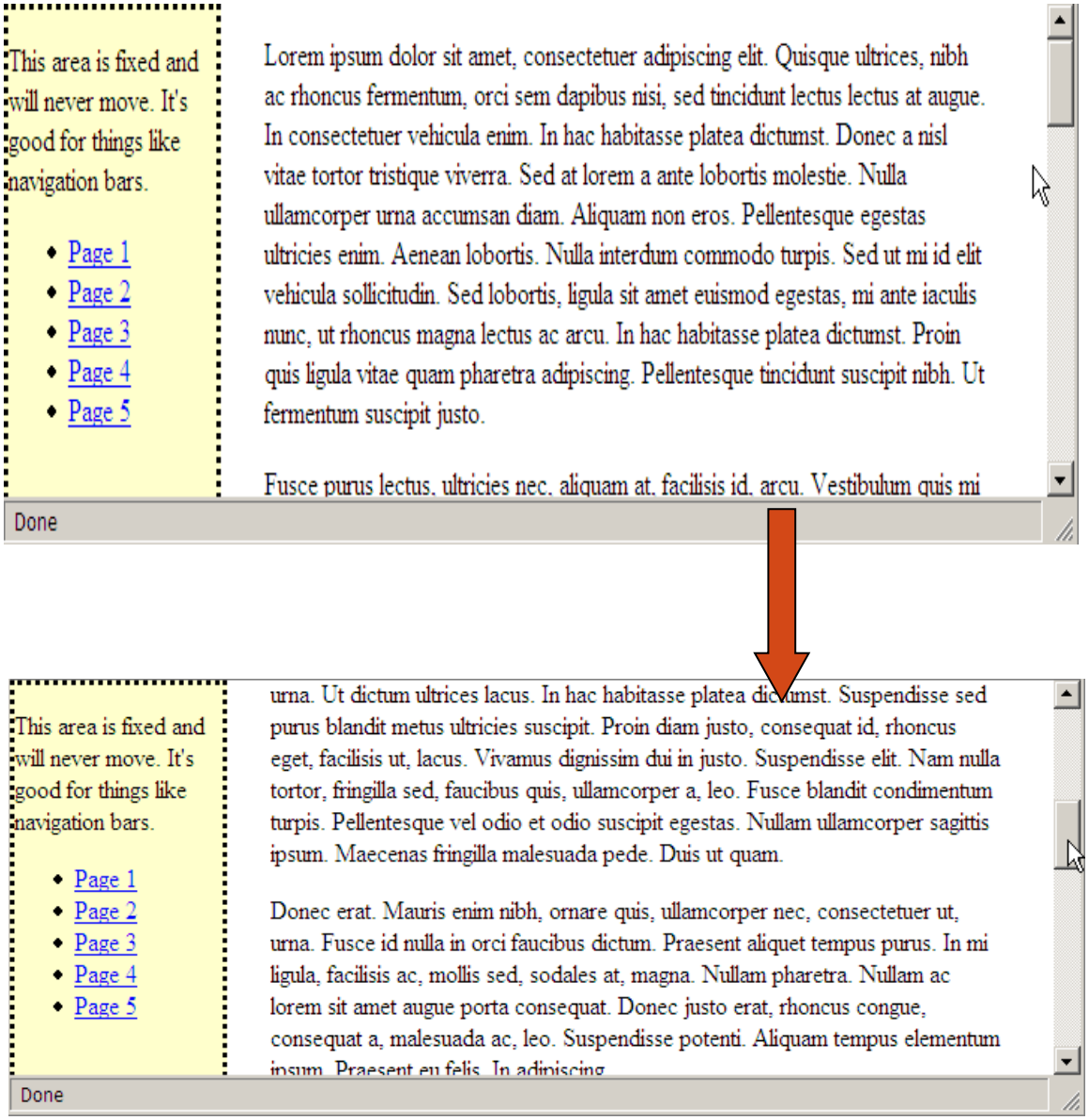
    <li><a href="page4.html">Page 4</a></li>

    <li><a href="page5.html">Page 5</a></li>

  </ul>

</div></body></html>
```

Example: position:fixed (in Firefox)



Relative Or Absolute Positioning

- type of positioning (relative or absolute, etc.) used by a particular style rule is determined by the position property
- **left**—The left position offset
- **right**—The right position offset
- **top**—The top position offset
- **bottom**—The bottom position offset

In relative positioning, content is displayed according to the flow of a page, with each element physically appearing after the element preceding it in the HTML code.

Absolute positioning, on the other hand, allows you to set the exact position of content on a page.

Example1: Relative Positioning with Four Color Blocks

```
<html>
<html
<head>
<title>Positioning the Color
Blocks</title>
<style type=”text/css”>
div {
position:relative;
width:250px;
height:100px;
border:5px solid #000;
color:black;
font-weight:bold;
text-align:center;
}
div#d1 {
background-color:red;
}
div#d2 {
background-color:green;
}
```

```
div#d3 {
background-color:blue;
}
div#d4 {
background-color:yellow;
}
</style>
</head>
<body>
<div id=”d1”>DIV #1</div>
<div id=”d2”>DIV #2</div>
<div id=”d3”>DIV #3</div>
<div id=”d4”>DIV #4</div>
</body>
</html>
```

Example1: Using Absolute Positioning of the Color Blocks

```
<html>
<head>
<title>Positioning the Color
Blocks</title>
<style type="text/css">
div {
position:absolute;
width:250px;
height:100px;
border:5px solid #000;
color:black;
font-weight:bold;
text-align:center;
}
div#d1 {
background-color:red;
left:0px;
Top:0px;
}
div#d2 {
background-color:green;
left:75px;
top:25px;}
```

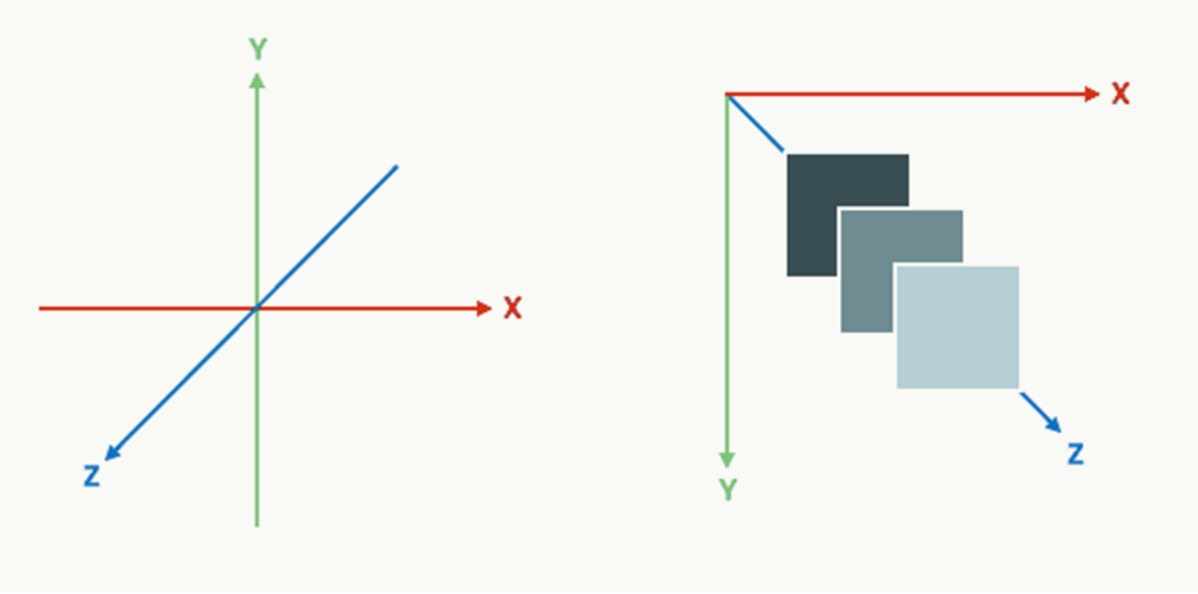
```
div#d3 {
background-color:blue;
left:150px;
top:50px;
}
div#d4 {
background-color:yellow;
left:225px;
top:75px;
}
</style>
</head>
<body>
<div id="d1">DIV #1</div>
<div id="d2">DIV #2</div>
<div id="d3">DIV #3</div>
<div id="d4">DIV #4</div>
</body>
</html>
```

Layers and the Bounding Box

- When the browser draws an object on a page, it places it into an invisible rectangular space called a "bounding box." You can set the box's exact location on the page or offset it from other objects on the page. As mentioned in the previous slides, you can also specify the size of the box.
- With CSS, these boxes can be stacked one on top of another as layers. Horizontal and vertical positioning happen along the X and Y axes, and the layered positioning happens along the Z axis.
- The Z axis is set using the CSS style `z-index`, which allows you to specify which layer appears on top of the others. By setting the `z-index` higher or lower, an object can move up and down a stack. The higher the `z-index`, the more "on top" it is.

Layers and the Bounding Box

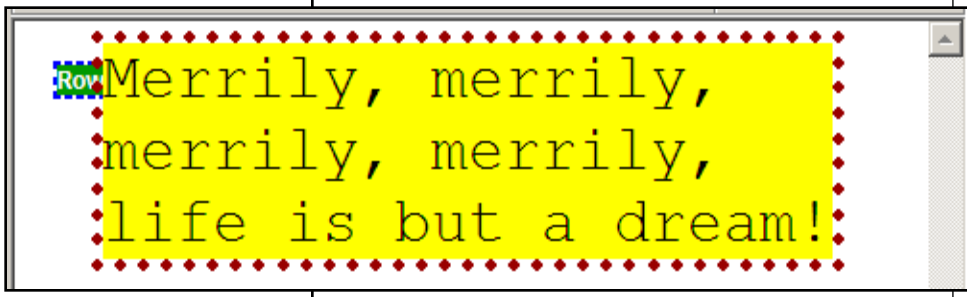
- To get layers overlapping each other you need to use the **z-index** command.
- The higher the **index**, the closer the div will appear to the front of the page.
- Z-index Illustration:



Layering Example 1

```
<head>
<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1" />
<title>Untitled Document</title>
<style type="text/css">
<!--
.over {
        font-family: "Courier New", Courier, mono;
        font-size: 200%;
        color: #000000;
        background-color: #FFFF00;
        border: thick dotted #990000;
        position: absolute;
        left: 40px;
        top: 5px;
        z-index: 2;
}
.under {
        font-family: Arial, Helvetica, sans-serif;
        font-size: 75%;
        font-weight: bold;
        color: #FFFFFF;
        background-color: #009900;
        border: thin dashed #0000FF;
        position: absolute;
        left: 20px;
        top: 20px;
        z-index: 1;
}
-->
</style>
</head>

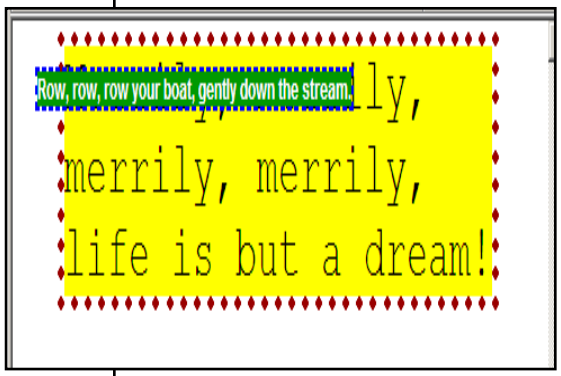
<body>
<p class="under">Row, row, row your boat, gently down the stream.</p>
<p class="over">Merrily, merrily, merrily, merrily, life is but a dream!</p>
</body>
```



Layering Example 2

```
<head>
<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1" />
<title>Untitled Document</title>
<style type="text/css">
<!--
.over {
        font-family: "Courier New", Courier, mono;
        font-size: 200%;
        color: #000000;
        background-color: #FFFF00;
        border: thick dotted #990000;
        position: absolute;
        left: 40px;
        top: 5px;
        z-index: 1;
}
.under {
        font-family: Arial, Helvetica, sans-serif;
        font-size: 75%;
        font-weight: bold;
        color: #FFFFFF;
        background-color: #009900;
        border: thin dashed #0000FF;
        position: absolute;
        left: 20px;
        top: 20px;
        z-index: 2;
}
-->
</style>
</head>

<body>
<p class="under">Row, row, row your boat, gently down the stream.</p>
<p class="over">Merrily, merrily, merrily, merrily, life is but a dream!</p>
</body>
```



Example1: z-index to Alter the Display of Elements in the Color Blocks Sample

```
<html>
<head>
<title>Positioning the Color Blocks</title>
<style type=”text/css”>
div {
position:absolute;
width:250px;
height:100px;
border:5px solid #000;
color:black;
font-weight:bold;
text-align:center;
}
div#d1 {
background-color:red;
left:0px;
top:0px;
z-index:0;
}
```

```
div#d2 {
background-color:green;
left:75px;
top:25px;
z-index:3;
}
div#d3 {
background-color:blue;
left:150px;
top:50px;
z-index:2;
}
```

```
div#d4 {
background-color:yellow;
left:225px;
top:75px;
z-index:1;
}
</style>
</head>
<body>
<div id=”d1”>DIV #1</div>
<div id=”d2”>DIV #2</div>
<div id=”d3”>DIV #3</div>
<div id=”d4”>DIV #4</div>
</body>
</html>
```

Practice

- Read the following case study and give reasons for asking certain question in relation to style sheets.
- A school wishes to design a website. The function of this website is to let its students know when assignments are due, the dates of exams, timetables, subjects taught, syllabuses and the teachers who teach them.
- The school wishes to have quality controls over the Web design, since the teachers themselves will be responsible for updating the information on the Web.

1. The school has approached you to advise them on how this may be possible.
2. The school wants you to explain the process you wish to use, and why it is better than letting every teacher write their own style.
3. Write a list of ten or more HTML tags and classes that may be used for the website. Consider the attributes that may have to be used, such as background colour, departmental style, tables (for timetable, exam, curriculum and annual), headers and so on.
4. After choosing the HTML tags, consider the CSS code required to style them.

Additional Resources

1. https://webdesign.tutsplus.com/articles/what-you-may-not-know-about-the-z-index-property--webdesign-16892
2. https://www.w3schools.com/cssref/pr_pos_z-index.asp